

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science Engineering and Information Systems

Fall Semester 2026

BCSE306L - Artificial Intelligence

Course Project

Class ID : VL2026270102517

Review - 2

Date: {{DD/MM/YYYY}}

Folio: A NotebookLM-Style Retrieval-Augmented Generation System

24BYB0050 – Swayam Prakash Yadav

24BYB0026 – Trinabha Dixit

Under the Guidance of

Prof. Nivedita S

Assistant Professor, SCORE

PROJECT SNAPSHOT

PART A – PROJECT OVERVIEW

Particular	Details
Project Title	Folio: A NotebookLM-Style Retrieval-Augmented Generation System
Area of Application	Education and knowledge work: an AI study assistant that answers questions from a student's own documents (natural language processing and information retrieval).
Scope	A multi-user web application that runs on one laptop with Docker Compose. Users add PDF, Word, PowerPoint, Markdown, text files or pasted notes to notebooks, ask questions answered only from those sources with clickable page, slide or passage citations, and generate quizzes, flashcards, study guides, FAQs, timelines and glossaries. Uses only free AI services (local embeddings and reranker, Groq free tier). Out of scope: web links, audio and video sources, sharing between users and multi-server deployment.

Alignment with Sustainable Development Goals

SDG No.	SDG Title	Justification
SDG 4	Quality Education	Helps students learn from their own course material: every answer is grounded in and cites the student's notes, slides and papers, and the study tools (quizzes, flashcards, study guides) support revision at no cost on an ordinary laptop.

Inputs – Features – Outputs

S. No.	Feature	Input Given	Output
1	Source ingestion	PDF, DOCX, PPTX, Markdown or text file, or pasted text	Indexed source: page text, structure-aware chunks, embeddings, a short summary and suggested questions
2	Cited question answering	A natural-language question (follow-ups use the chat history)	Streamed answer in which every claim carries a numbered citation that opens the exact page, slide or passage
3	Retrieval Inspector	Any answer	Rewritten query, ranked candidate passages with vector, keyword and reranker scores, confidence and per-stage timings
4	Study tools and Studio	Chosen sources, optional topic, item count and quiz difficulty	Quiz, flashcards (Anki/CSV export), study guide, FAQ, timeline or glossary, every item cited
5	Notes	An answer to save, or text typed by the user	Saved notes that keep working citation chips
Final Output		A grounded, verifiable study assistant: answers and study material drawn only from the user's documents, with every claim traceable to its source page.	

Demo Video

Particular	Details
Video URL	

Particular	Details
Duration	

Technical Stack

Layer	Technology / Tool
Frontend	React 19, TypeScript, Vite, Tailwind CSS 4, TanStack Query, Zustand, Radix UI, react-pdf (pdf.js), KaTeX
Backend	Python 3.13, FastAPI, async SQLAlchemy, Alembic migrations, Pydantic, Server-Sent Events, uv
Database	PostgreSQL 16 with pgvector (HNSW cosine index) and PostgreSQL full-text search
AI / ML Models & Libraries	BAAI/bge-small-en-v1.5 embeddings (sentence-transformers), cross-encoder/ms-marco-MiniLM-L6-v2 reranker, Groq openai/gpt-oss-120b (answers) and gpt-oss-20b (evaluation judge), RapidOCR, PyMuPDF, python-docx, python-pptx, PyTorch (CPU)
Tools & IDE	Visual Studio Code, Claude Code (AI coding assistant), Git and GitHub, Docker Desktop, Playwright, npm
Deployment / Hosting	Docker Compose on a local laptop: nginx (frontend), FastAPI (backend) and PostgreSQL containers; no cloud hosting

Architecture Diagram

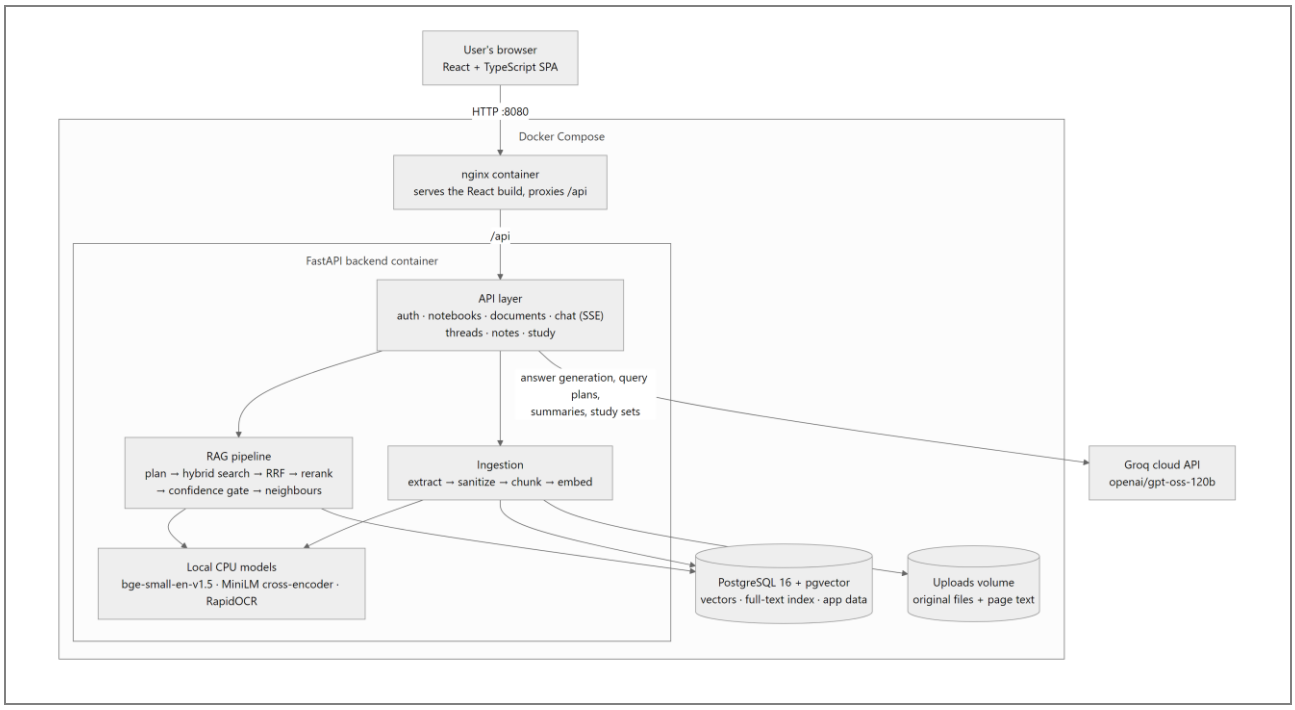


Fig. 1: System architecture of Folio

PART B – TEAM AND PROJECT JOURNEY

Particular	Details
Project Type	Multi-member (2 students)

Division of Work and Skills

S. No.	Student Name	Register Number	Work Done	Technical Skills Already Known	Skills, Tools & Knowledge Acquired
1	Swayam Prakash Yadav	24BYB0050	Both members worked across the whole system: document ingestion and chunking, hybrid retrieval and reranking, cited answer generation, the React interface, study tools, evaluation harness, testing and this report.	Python, JavaScript, HTML/CSS, SQL, Git	FastAPI, React with TypeScript, PostgreSQL with pgvector, sentence embeddings and cross-encoders, retrieval-augmented generation, prompt design, Docker, retrieval evaluation (Hit@k, MRR)
2	Trinabha Dixit	24BYB0026	Both members worked across the whole system: document ingestion and chunking, hybrid retrieval and reranking, cited answer generation, the React interface, study tools, evaluation harness, testing and this report.	Python, JavaScript, HTML/CSS, SQL, Git	FastAPI, React with TypeScript, PostgreSQL with pgvector, sentence embeddings and cross-encoders, retrieval-augmented generation, prompt design, Docker, retrieval evaluation (Hit@k, MRR)

Phases and Plan

Particular	Details
Number of Phases Planned	8

Phase	Activities	Timeline (From – To)	Status
Phase 1: Requirements and design	Define the core features from NotebookLM; choose the stack (FastAPI, pgvector, local embeddings, Groq); write the design specification	06/10/2026 – 06/10/2026	Completed
Phase 2: Core backend	Username/password auth with JWT cookies and login rate limiting, notebooks, PDF upload with background processing, database migrations	06/10/2026 – 06/10/2026	Completed
Phase 3: Cited chat and frontend	Streamed answers over SSE with [[Cn]] citation markers; React workspace with PDF viewer and passage highlighting; Docker packaging	06/10/2026 – 06/10/2026	Completed
Phase 4: Hybrid search, OCR and study tools	Vector and keyword search fused with RRF, OCR for scanned pages, quizzes and flashcards	06/10/2026 – 06/10/2026	Completed
Phase 5: Retrieval upgrade and evaluation	Review-1 feedback: query rewriting, cross-encoder reranking, confidence gate, structure-aware chunking, content security, evaluation harness, redesigned interface	07/10/2026 – 19/10/2026	Completed

Phase	Activities	Timeline (From – To)	Status
Phase 6: New source types and RAG tuning	Word, PowerPoint, Markdown, text and pasted sources; tables; source summaries; neighbouring-chunk context; answer cache; threshold tuning; evaluation re-run	20/10/2026 – 20/10/2026	Completed
Phase 7: Workspace features	Chat threads, regenerate, notes, Studio outputs, quiz difficulty and mistake review, Anki/CSV export, PDF viewer toolbar, live indexing progress	20/10/2026 – 20/10/2026	Completed
Phase 8: UI polish, testing and documentation	UI bug fixes, phone and projector layouts, browser verification, README and this report	20/10/2026 – ongoing	In progress

Difficulties Faced in Implementation

S. No.	Difficulty Faced	Where It Went Wrong	How It Was Corrected
1	Citation relevance showed 0.00 for almost every source	The cross-encoder outputs sharply polarised probabilities; for summary-type questions every passage scored about 0.0001, and rounding to four decimals turned them into 0.00	Citations now show the citation-check support score and the embedding question-match score; scores keep four significant digits and the Inspector draws reranker scores on a logit scale
2	Correctly cited answers were flagged as uncited	The sentence splitter separated a marker placed after the full stop ("... bleached. [[C1]]") from its claim, so coverage read 0% and good answers received a warning	A marker-only sentence now credits the claim just before it; fixed edge cases were checked by hand
3	Code listings in PDFs were treated as tables	PyMuPDF's table detector reported shaded code boxes as one-row tables, which would have replaced code with broken Markdown tables	Only real grids are accepted: at least 2 x 2 cells, at least 60% filled, and not entirely monospaced text
4	Groq free-tier rate limits (about 8,000 tokens per minute)	Summaries, study sets and evaluation runs issued back to back returned HTTP 429	Retry-after countdown in the UI, summaries never block indexing, the evaluation harness paces its calls, and repeat questions are served from the answer cache
5	Citation highlights misaligned after PII masking	Masking was applied to each chunk after chunking, so the stored offsets no longer matched the text	Masking and normalisation now run on the page blocks before chunking, so chunk offsets, stored page text and highlights always agree

Review History

Particular	Details
Number of Reviews Attended	1

Review No.	Review Date	Feedback Received	How It Was Addressed
Review 1	{{DD/MM/YYYY}}	Test various types of chunking, embedding and encoding.	Compared the fixed sliding-window chunker with a new structure-aware chunker (headings, paragraphs, code, tables); embedded each chunk together with its section title using BGE's query instruction; added a cross-encoder reranker over the bi-encoder results; built an evaluation harness that measures each change (Hit@1 77% to 93%, MRR 0.852 to 0.948).

Pitch and Outcomes

Particular	Details
Where Can the Project Be Pitched	Not applicable

Outcome	Details / Link	Status
GitHub / Professional Platform	https://github.com/swayam5342/Folio	Published (public repository)
Patent	Not applicable	Not applicable
Publication	Not applicable	Not applicable
Startup	Not applicable	Not applicable
Any Other Plan	Not applicable	Not applicable

Status: Planned / In progress / Filed / Published / Not applicable

TABLE OF CONTENTS

ABSTRACT 9
1. INTRODUCTION 9
1.1 Background 9
1.2 Motivation 9
2. PROBLEM STATEMENT 9
3. OBJECTIVES 10
4. LITERATURE SURVEY 10
4.1 Comparison of Related Work 10
4.2 Key Findings 11
4.3 Research Gap 11
5. PROPOSED SYSTEM 11
6. METHODOLOGY 12
6.1 Approach 12
6.2 Dataset Description 12
7. REQUIREMENTS 13
7.1 Functional Requirements 13
7.2 Non-Functional Requirements 13
7.3 Hardware and Software Requirements 14
8. SYSTEM DESIGN 14
8.1 Detailed Design 14
8.2 UML Diagrams 15
9. IMPLEMENTATION 18
9.1 Key Modules 18
9.2 Screenshots 19
10. TESTING AND RESULTS 24
10.1 Test Cases 24
10.2 Performance Metrics 25
10.3 Discussion of Results 26
11. CONCLUSION AND FUTURE WORK 26
11.1 Conclusion 26
11.2 Future Work 26
REFERENCES 27
APPENDIX – SOURCE CODE 27

LIST OF FIGURES

Fig. No.	Caption	Page No.
Fig. 1	System architecture of Folio	3
Fig. 2	Use case diagram	15
Fig. 3	Sequence diagram for answering a question	16
Fig. 4	Class diagram of the data model	17
Fig. 5	Activity diagram of source ingestion	18
Fig. 6	Notebook overview with suggested questions in a new chat	19
Fig. 7	Cited answer with the PDF opened at the cited page and passage highlighted	20
Fig. 8	Citation preview with support and question-match scores	20
Fig. 9	Retrieval Inspector: confidence, stage timings and ranked passages	21
Fig. 10	PowerPoint citation opened in the text reader at the cited slide	21
Fig. 11	Study guide generated from the sources, every section cited	22
Fig. 12	Quiz with feedback, explanation and citation	22
Fig. 13	Answer saved to Notes with a working citation	23
Fig. 14	Live indexing progress for an 80-page PDF	23
Fig. 15	Workspace on a phone-sized screen	24
Fig. 16	Retrieval quality by configuration (evaluation harness)	26

LIST OF TABLES

Table No.	Caption	Page No.
Table 1	Comparison of related work	10
Table 2	Evaluation dataset	12
Table 3	Hardware and software requirements	14
Table 4	Key modules	18
Table 5	Test cases	24
Table 6	Performance metrics of the final pipeline	25

ABSTRACT

Large language models answer fluently but cannot see a student's own notes, and when information is missing they may invent plausible text. Folio is a NotebookLM-style web application that answers questions only from the documents a user uploads and ties every claim to the page, slide or passage it came from. Sources (PDF, Word, PowerPoint, Markdown, plain or pasted text) are parsed into structured blocks, split into section-aware chunks and embedded locally with BAAI/bge-small-en-v1.5 into PostgreSQL with pgvector. At question time a follow-up is rewritten into a standalone query, vector and keyword results are fused with reciprocal rank fusion, a local cross-encoder reranks the candidates and a confidence gate declines questions the sources cannot answer. The selected passages, with their neighbouring chunks, are sent to openai/gpt-oss-120b on Groq's free tier with numbered markers; the streamed answer's citations are parsed into clickable chips and checked against their excerpts. The system also generates cited quizzes, flashcards, study guides, FAQs, timelines and glossaries. On a 50-question evaluation set, the full pipeline raised Hit@1 from 77% (vector search alone) to 93% and MRR from 0.852 to 0.948, with 96% citation precision and 97% faithfulness, while running entirely on a laptop at no cost.

Keywords: Retrieval-Augmented Generation, Hybrid Search, Cross-Encoder Reranking, Citation Grounding

1. INTRODUCTION

1.1 Background

Large language models (LLMs) can summarise, explain and answer questions in natural language, but their knowledge is fixed at training time and they have no access to a user's private documents. When asked about material they have not seen they may produce confident but unsupported text, a failure known as hallucination.

Retrieval-augmented generation (RAG) addresses this by retrieving relevant passages from a document collection and giving them to the model as context, so that answers are grounded in real text [1]. Google's NotebookLM [8] applied the idea to personal study material: users upload sources into a notebook and receive answers with citations. Folio rebuilds the core of that experience as an open, self-hosted system that a student can run on a laptop.

1.2 Motivation

Students keep their course material in many forms (lecture PDFs, slide decks, Word handouts, typed notes) and need quick, trustworthy answers from it while revising. General chatbots cannot see this material, and even a correct answer is hard to trust when the reader cannot check where it came from.

Hosted tools such as NotebookLM are closed and depend on a vendor account. This project set out to show that a comparable, verifiable assistant can be built from open components and free services, and to measure, rather than assume, how much each retrieval technique improves answer quality.

2. PROBLEM STATEMENT

Given a private collection of a student's documents in mixed formats, answer natural-language questions using only that collection, attach to every claim a citation that opens the exact supporting page, slide or passage, and decline when the documents do not contain the answer. The system must keep each user's data private, run on a single laptop using free AI services, handle scanned and hostile files safely, and support revision through cited study material.

3. OBJECTIVES

- Let a user add PDF, Word, PowerPoint, Markdown, text or pasted sources to separate notebooks and ask questions about them.
- Ground every answer in the retrieved text and decline plainly when the sources do not contain the answer.
- Attach page-, slide- or passage-level citations that open the original source with the cited text highlighted.
- Improve retrieval with hybrid search, query rewriting, cross-encoder reranking and structure-aware chunking, and measure each stage on an evaluation set.
- Generate cited study material: quizzes, flashcards, study guides, FAQs, timelines and glossaries.
- Keep each user's data private and run entirely on free services with one Docker Compose command.

4. LITERATURE SURVEY

4.1 Comparison of Related Work

Table 1: Comparison of related work

S. No.	Paper Title and Year	Methods Used	Advantages	Limitations	Research Gap	How This Project Addresses the Gap
1	Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (2020) [1]	Dense retriever (DPR) combined with a BART generator, trained end to end over a Wikipedia index	Grounds generation in retrieved text; knowledge can be updated without retraining the generator	Requires training; no explicit citations; evaluated on Wikipedia, not personal documents	Users cannot verify which passage supports each claim in their own documents	Numbered [[Cn]] markers mapped to file, page or slide; works on uploaded files without any training
2	Dense Passage Retrieval for Open-Domain Question Answering (2020) [2]	Bi-encoder that embeds questions and passages separately; nearest-neighbour search	Semantic matching that outperforms BM25 on paraphrased questions	Misses exact identifiers such as course codes and function names	Dense retrieval alone fails on keyword-style queries	Hybrid vector + PostgreSQL full-text search fused with reciprocal rank fusion [6]
3	Passage Re-ranking with BERT (2019) [3]	Cross-encoder that reads the query and passage together to score relevance	Much higher ranking precision than bi-encoder retrieval	Too slow to score a whole collection; scores are sharply polarised	Precision versus cost on modest hardware	Reranks only the top 30 fused candidates on CPU; displays support scores instead of raw probabilities

S. No.	Paper Title and Year	Methods Used	Advantages	Limitations	Research Gap	How This Project Addresses the Gap
4	C-Pack: Packed Resources for General Chinese Embeddings (BGE) (2024) [4]	Contrastively trained embedding models with an instruction prefix for retrieval queries	Small, strong open embedding model (384 dimensions for the small English model)	General purpose; a chunk loses the context of the section it came from	Context-free chunks retrieve poorly for headings and follow-ups	Structure-aware chunks embedded together with their section title; BGE query instruction applied
5	Enabling Large Language Models to Generate Text with Citations (2023) [5]	Benchmark (ALCE) and prompting methods that make LLMs cite retrieved passages	Defines automatic measures of citation quality	Citations point to passages, not locations a reader can open; no repair of weak citations	Citations that readers can check in the original document	Per-claim support check with a cross-encoder, citation coverage measure, and highlighting at the cited location

4.2 Key Findings

Retrieval quality bounds answer quality: when the right passage is not retrieved the model either hallucinates or declines, so most improvement effort belongs in retrieval [1], [2].

Dense and lexical retrieval fail in different ways, and fusing their rankings with reciprocal rank fusion is a simple, robust way to get the benefits of both [2], [6].

Cross-encoder reranking gives large precision gains when it is applied only to a small candidate set [3], and sentence embeddings make the first-stage search fast enough for interactive use [4], [7].

Citation quality must be measured separately from fluency; models often cite passages that do not support the claim [5].

4.3 Research Gap

Existing work is mostly evaluated on Wikipedia-scale benchmarks with trained models. There is little guidance for a student-scale system that must run on a laptop with free services, handle mixed document formats, and make every citation openable at its exact location. Folio fills this gap by combining hybrid search, local reranking, structure-aware chunking and citation verification in one self-hosted application, and by measuring each stage's contribution.

5. PROPOSED SYSTEM

Folio is a three-container web application (Fig. 1). An nginx container serves the React single-page interface and forwards /api requests to a FastAPI backend; the backend stores users, notebooks, documents, chunks with their 384-dimensional vectors, chat threads, notes and study sets in PostgreSQL with pgvector, keeps the original files on a volume, and calls Groq only to write text.

A user's work is organised into notebooks. Each notebook holds sources, several chat threads, saved notes and study sets. Uploaded sources are processed in the background with live progress; each becomes searchable chunks plus a short summary and suggested questions. Questions are answered by the retrieval pipeline described in Section 6, and answers stream back with numbered citation chips. Clicking a chip opens the PDF

at the cited page, or the text reader at the cited slide or passage, with the supporting text highlighted. A Retrieval Inspector shows how each answer was found.

Every optional stage (hybrid search, query rewrite, multi-query, reranker, abstention, neighbour expansion, answer cache, summaries) has a feature flag, so the evaluation harness can switch stages off one at a time and measure what each adds.

6. METHODOLOGY

6.1 Approach

Ingestion. A file is validated by extension and content (deep PDF parse; zip-bomb and macro checks for Office files; binary detection for text), stored, and processed in the background. Extractors turn every format into pages of labelled blocks: PDF via PyMuPDF with fonts used to detect headings and code, PyMuPDF table detection and RapidOCR for scanned pages; PowerPoint slides become pages; Word, Markdown and text become a single page with headings. Blocks are normalised and optionally PII-masked before chunking, then split by a structure-aware chunker (about 800 characters, paragraphs never cut, tables and code kept whole). A deterministic scanner quarantines prompt-injection passages, and each chunk is embedded with its section title (Fig. 5).

Retrieval. A small JSON call rewrites a follow-up into a standalone query. The query is embedded with BGE's search instruction; a vector search (pgvector, cosine) and a full-text search run over the selected sources, and their rankings are fused with reciprocal rank fusion into 30 candidates. A cross-encoder reranks them; a confidence gate declines when both the reranker score and the embedding similarity are weak. The top six passages are chosen with de-duplication and a per-document cap, and the chunks before and after each one (same page and section) are attached.

Generation and verification. The passages are sent to the LLM labelled C1 to C6 with their file, page or slide and section, inside delimiters that mark them as untrusted data. The model must cite every claim as [[Cn]]. The answer streams over Server-Sent Events; markers are parsed into structured sources, citation coverage is measured, and a cross-encoder checks that each cited passage supports its claim. Clean answers are stored in an answer cache keyed by the standalone question and the notebook's indexed sources (Fig. 3).

Evaluation. A harness runs 50 questions through the real pipeline, adding one stage at a time, and reports Hit@k, MRR, abstention, citation precision, faithfulness and key-fact recall, with answers judged by a second model.

6.2 Dataset Description

Table 2: Evaluation dataset

Particular	Details
Source	Purpose-written evaluation corpus in the repository: four sample PDFs (TCP congestion control, OS virtual memory, a campus library policy and a sleep study) and backend/eval/dataset.jsonl
Size	50 questions (43 answerable, 7 unanswerable) over 20 PDF pages; answerable questions are 18 factual, 9 keyword, 11 paraphrase and 5 follow-up
Attributes / Features	Question id, type, document, question text, prior chat turns (follow-ups), gold page numbers and the key facts a complete answer must state

Particular	Details
Preprocessing	PDFs parsed with PyMuPDF, normalised, chunked (fixed 800/150 characters or structure-aware) and embedded with bge-small-en-v1.5; each configuration retrieves for every question; answers of the final configuration are judged by gpt-oss-20b

7. REQUIREMENTS

7.1 Functional Requirements

- Users can register, sign in and sign out; each user sees only their own notebooks, sources, chats and notes.
- Users can add PDF, DOCX, PPTX, Markdown and text files (up to 50 MB) or pasted text, re-index and delete sources.
- The system answers questions from the selected sources only, streams the answer and cites every claim.
- Clicking a citation opens the source at the cited page, slide or passage with the text highlighted.
- The system declines questions that the sources cannot answer and shows the closest passages instead.
- Users can keep several chat threads, regenerate answers, copy answers with their sources and save them to notes.
- Users can generate cited quizzes, flashcards, study guides, FAQs, timelines and glossaries, and export flashcards.
- The Retrieval Inspector shows the queries, candidate passages, scores, confidence and timings for each answer.

7.2 Non-Functional Requirements

- Privacy and security: hashed passwords, httpOnly session cookies, login rate limiting, ownership checks (other users' resources return 404), hostile-file rejection and prompt-injection quarantine.
- Cost: zero; embeddings, reranking and OCR run locally on the CPU and only text generation calls Groq's free tier.
- Performance: retrieval completes in about 2 seconds on a laptop CPU; answers stream so the first words appear quickly; repeat questions return from the cache.
- Reliability: rate limits and network failures produce clear messages with retry; jobs interrupted by a restart are marked failed instead of hanging.
- Usability: responsive layout for laptops, projectors and phones; Light, Dark and Aurora themes with a Calm mode; keyboard access.
- Portability: the whole system starts with one command, docker compose up --build.

7.3 Hardware and Software Requirements

Table 3: Hardware and software requirements

Category	Requirement
Hardware	Development and test machine: ASUS ROG Strix 16 laptop, Intel Core i7-13650HX, 16 GB RAM, NVIDIA GeForce RTX 4060 8 GB, Windows 11 Home. The system runs CPU-only; the GPU is not required.
Software	Docker Desktop with Docker Compose; Python 3.13 with uv; Node.js and npm (frontend build); PostgreSQL 16 with pgvector (container); a modern web browser; a Groq API key (free tier).

8. SYSTEM DESIGN

8.1 Detailed Design

Refer to Fig. 1 in the Project Snapshot. The backend is organised in layers. Routers (auth, notebooks, documents, chat, threads, notes, study, evaluation) validate input with Pydantic and check ownership through shared dependencies. Services hold the logic: extractors and the PDF parser, chunking, embeddings, retrieval, reranker, query planner, the retrieval pipeline, the LLM client, citation checking, content security, summaries, the answer cache and study-set generation. SQLAlchemy models map to PostgreSQL tables (Fig. 4) and Alembic migrations evolve the schema.

The frontend is a React single-page application with a three-pane workspace: sources on the left; chat, study or notes in the centre; and the viewer, Retrieval Inspector or citation list on the right. TanStack Query manages server data, Zustand holds workspace state, and answers arrive as a Server-Sent Events stream of stage, retrieval, token and done events. Use cases and the question-answering flow are shown in Fig. 2 and Fig. 3.

8.2 UML Diagrams

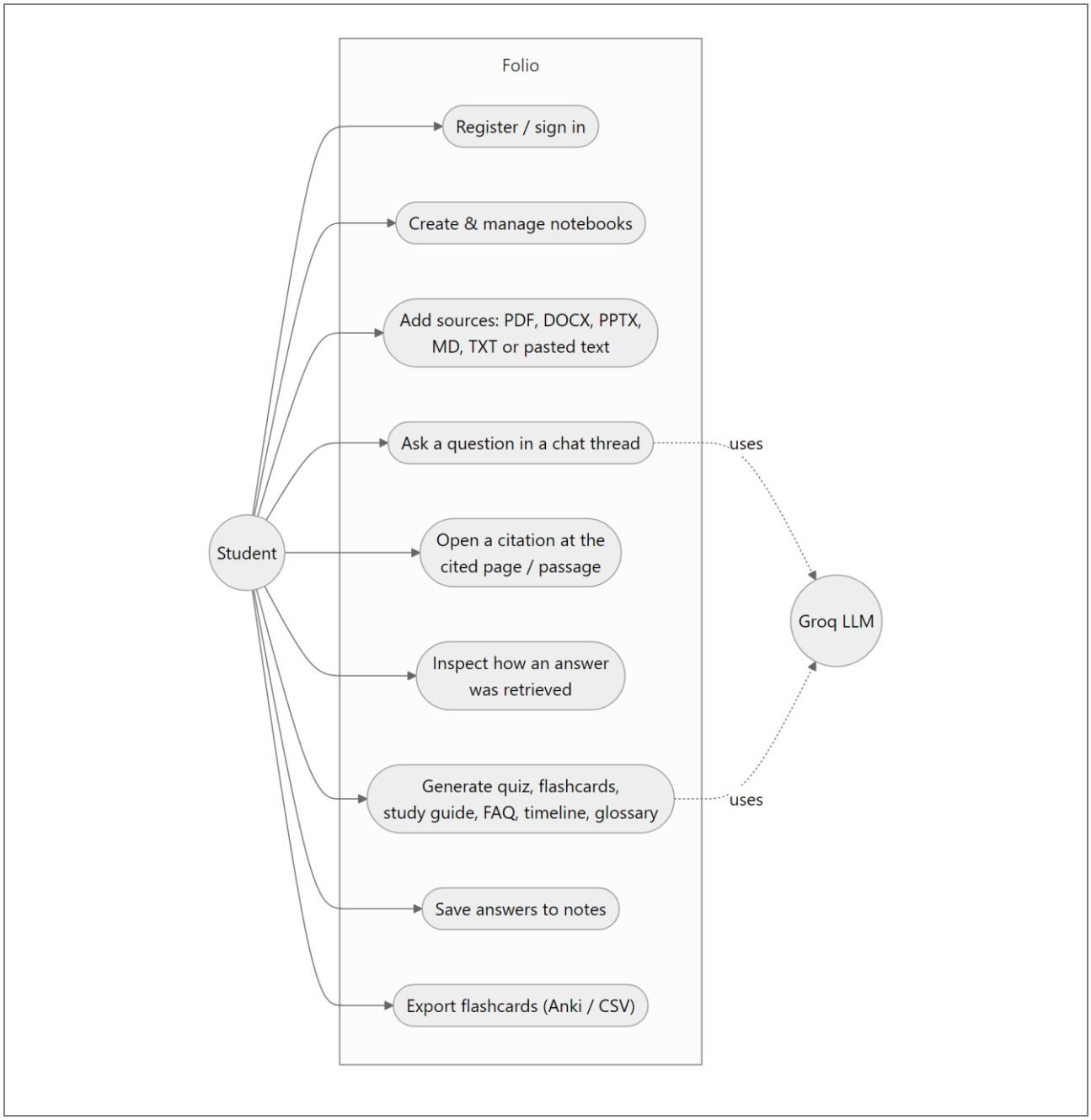


Fig. 2: Use case diagram

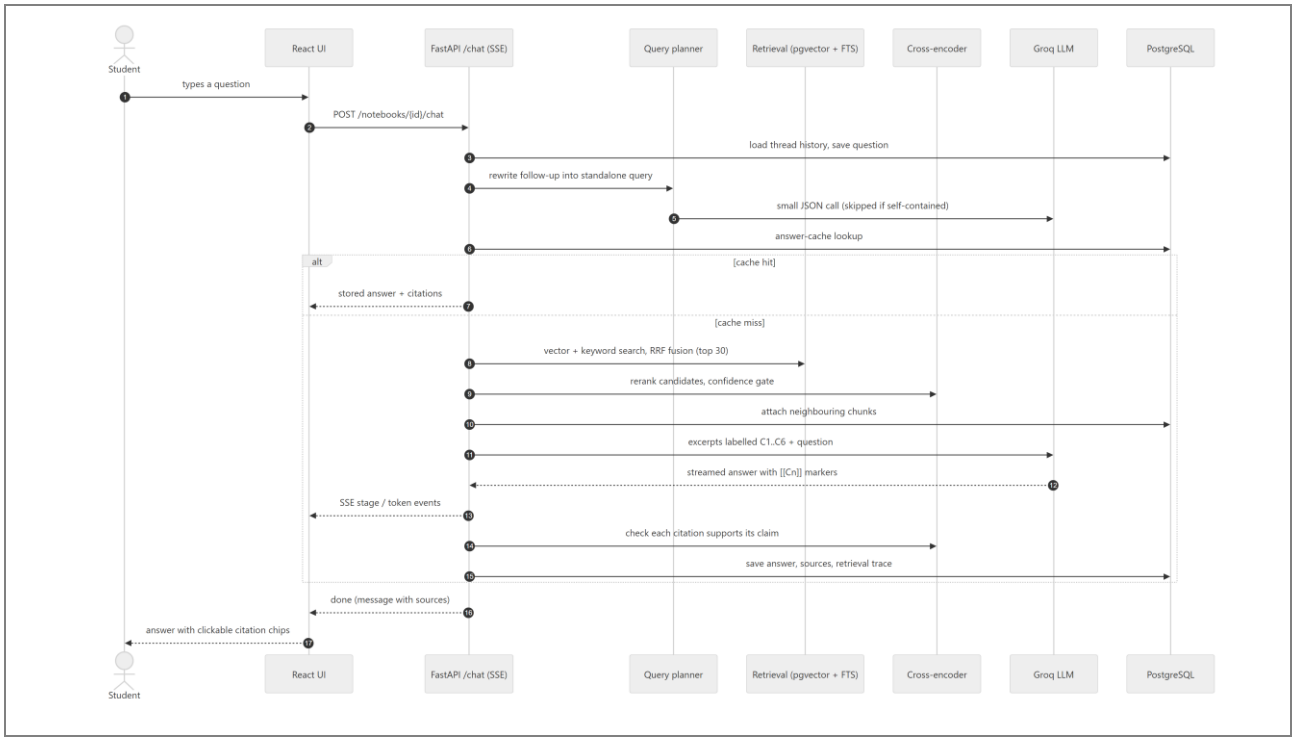


Fig. 3: Sequence diagram for answering a question

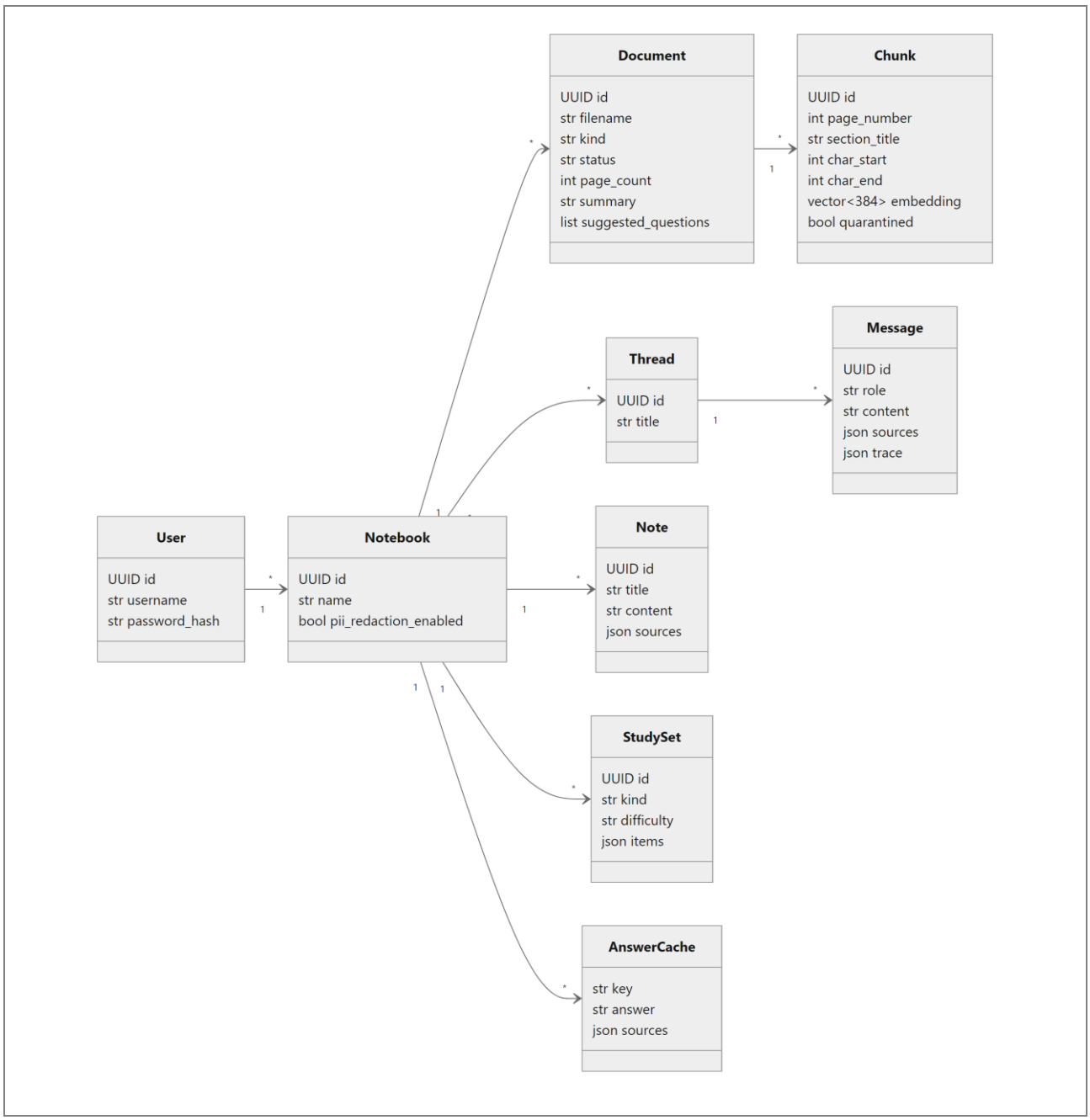


Fig. 4: Class diagram of the data model

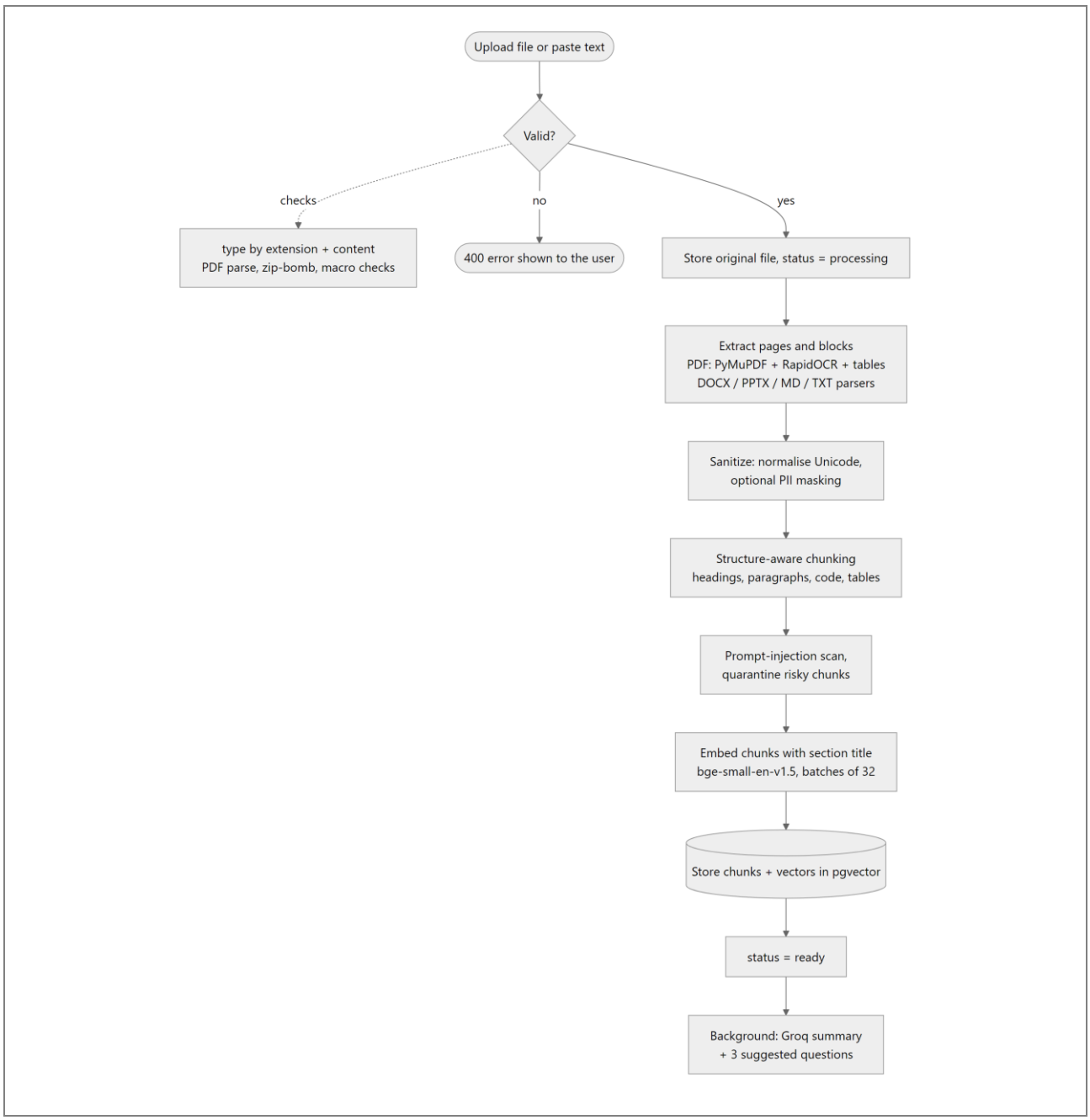


Fig. 5: Activity diagram of source ingestion

9. IMPLEMENTATION

9.1 Key Modules

Table 4: Key modules

S. No.	Module	Description
1	Extractors and PDF parser	Turn PDF, DOCX, PPTX, Markdown and text into pages of heading, body, code and table blocks; OCR for scanned pages; upload validation
2	Chunking	Structure-aware chunks of about 800 characters with section titles and character offsets; fixed sliding window as the baseline
3	Embeddings	bge-small-en-v1.5 vectors for chunks (with section title) and queries (with search instruction), in batches

S. No.	Module	Description
4	Retrieval	pgvector cosine search and PostgreSQL full-text search per query, fused with reciprocal rank fusion
5	Query planner	One small LLM call that rewrites follow-ups into standalone queries (optional paraphrases and HyDE)
6	Reranker and pipeline	Cross-encoder reranking, confidence gate, context selection and neighbouring-chunk expansion
7	LLM and citations	Prompt with labelled excerpts, Groq streaming, [[Cn]] parsing, citation coverage and support check
8	Content security	PDF/Office validation, Unicode normalisation, PII masking, prompt-injection quarantine
9	Summaries and answer cache	Per-source summary and suggested questions; cached answers for repeat questions
10	Study and Studio	Cited quizzes, flashcards, study guides, FAQs, timelines and glossaries with validation
11	Workspace UI	Sources, chat threads, notes, study players, PDF and text viewers, Retrieval Inspector

9.2 Screenshots

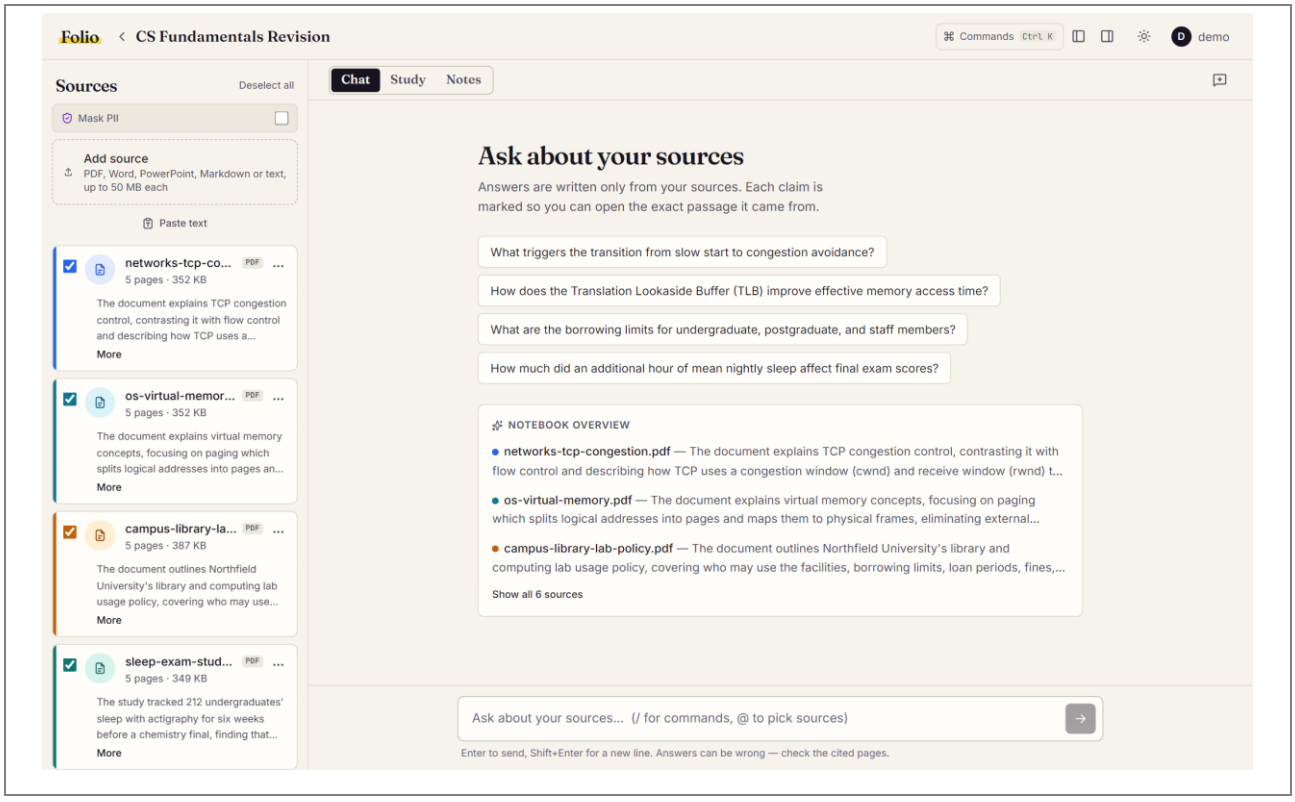


Fig. 6: Notebook overview with suggested questions in a new chat

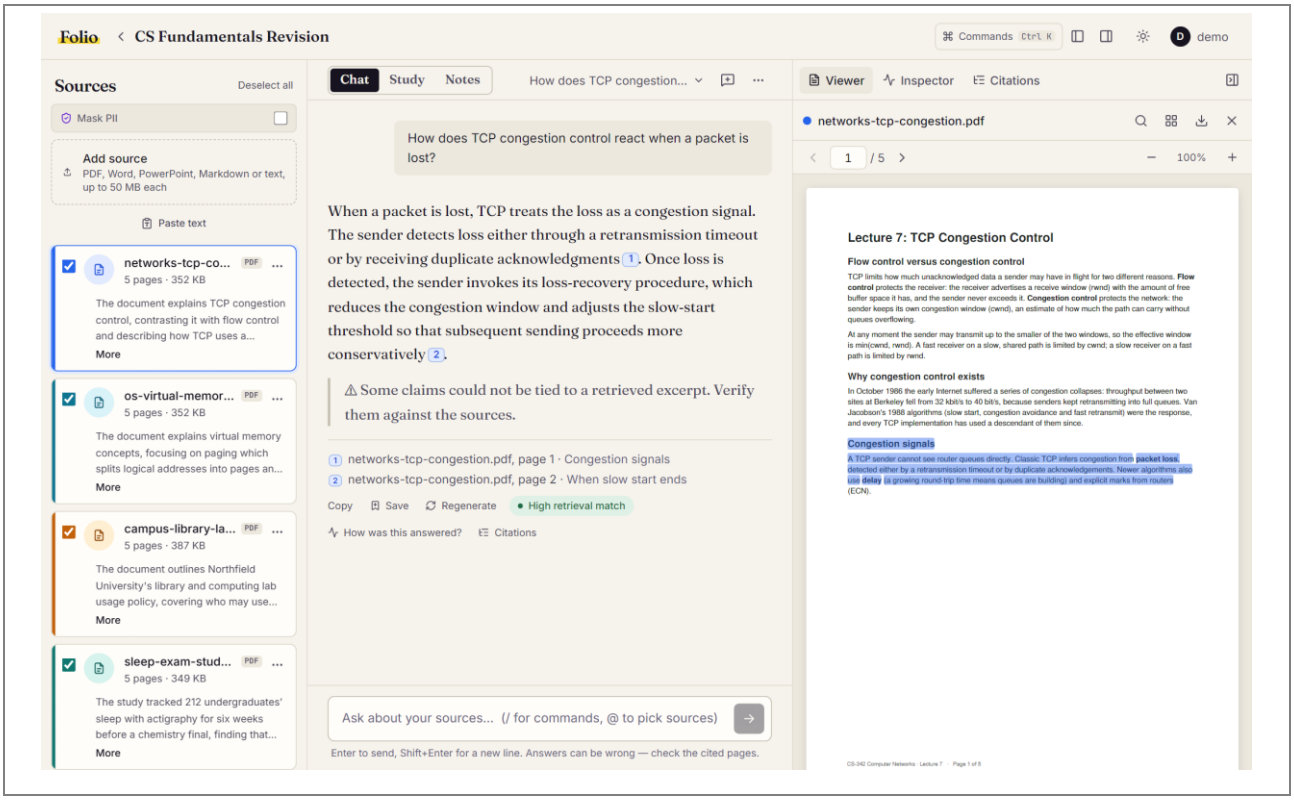


Fig. 7: Cited answer with the PDF opened at the cited page and passage highlighted

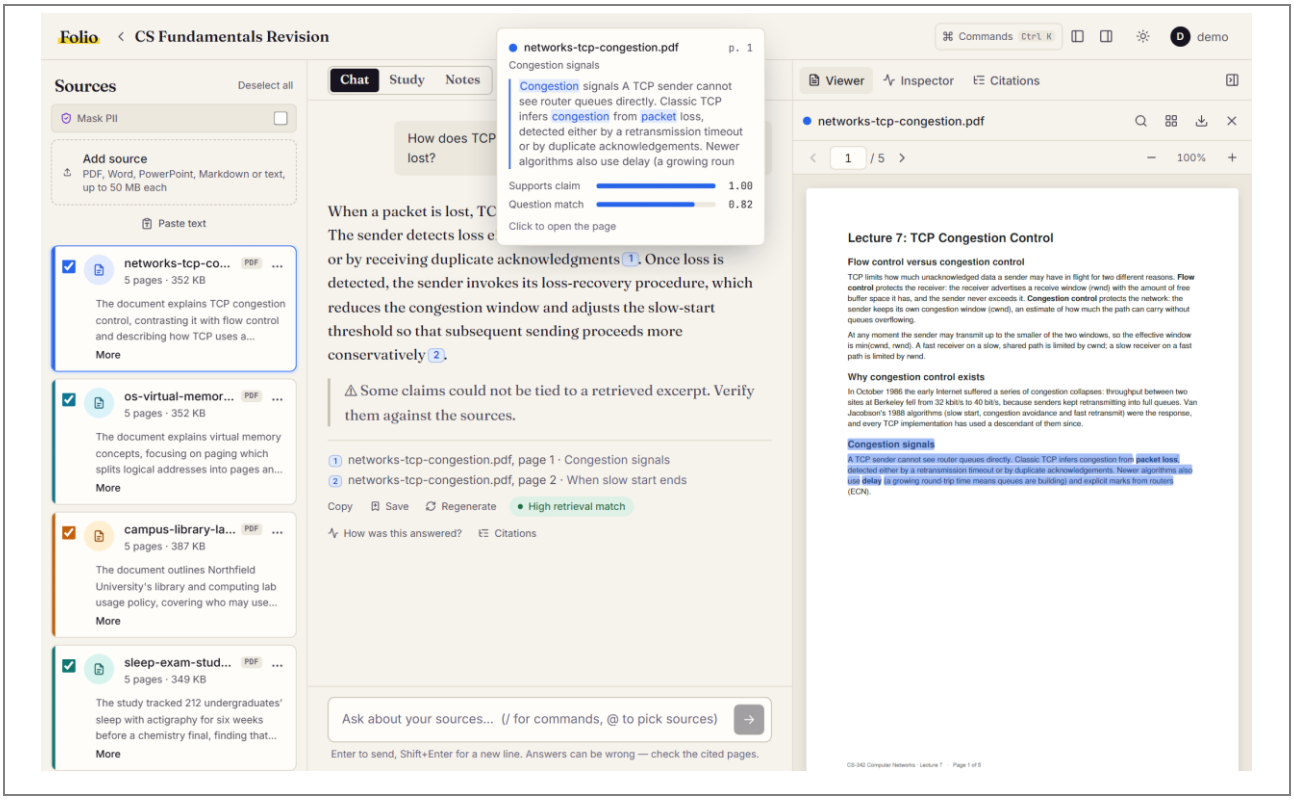


Fig. 8: Citation preview with support and question-match scores

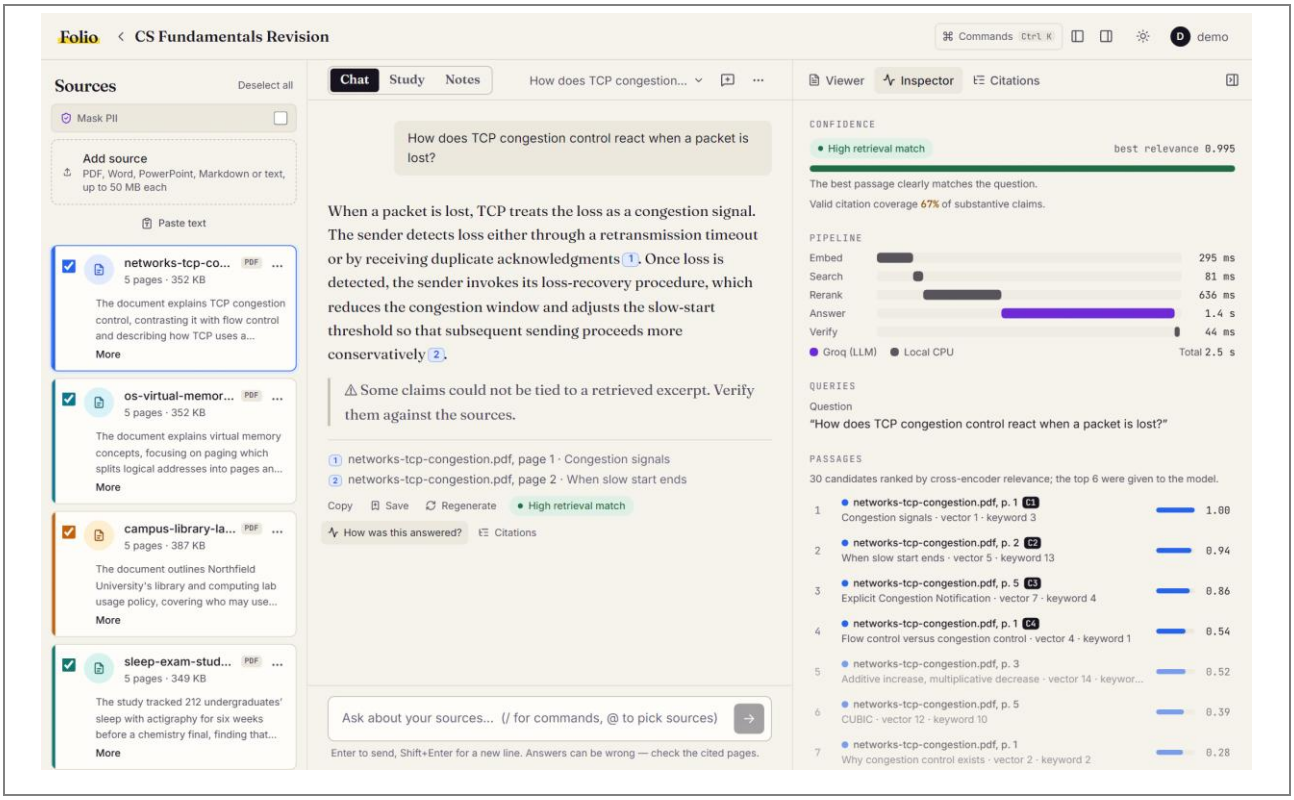


Fig. 9: Retrieval Inspector: confidence, stage timings and ranked passages

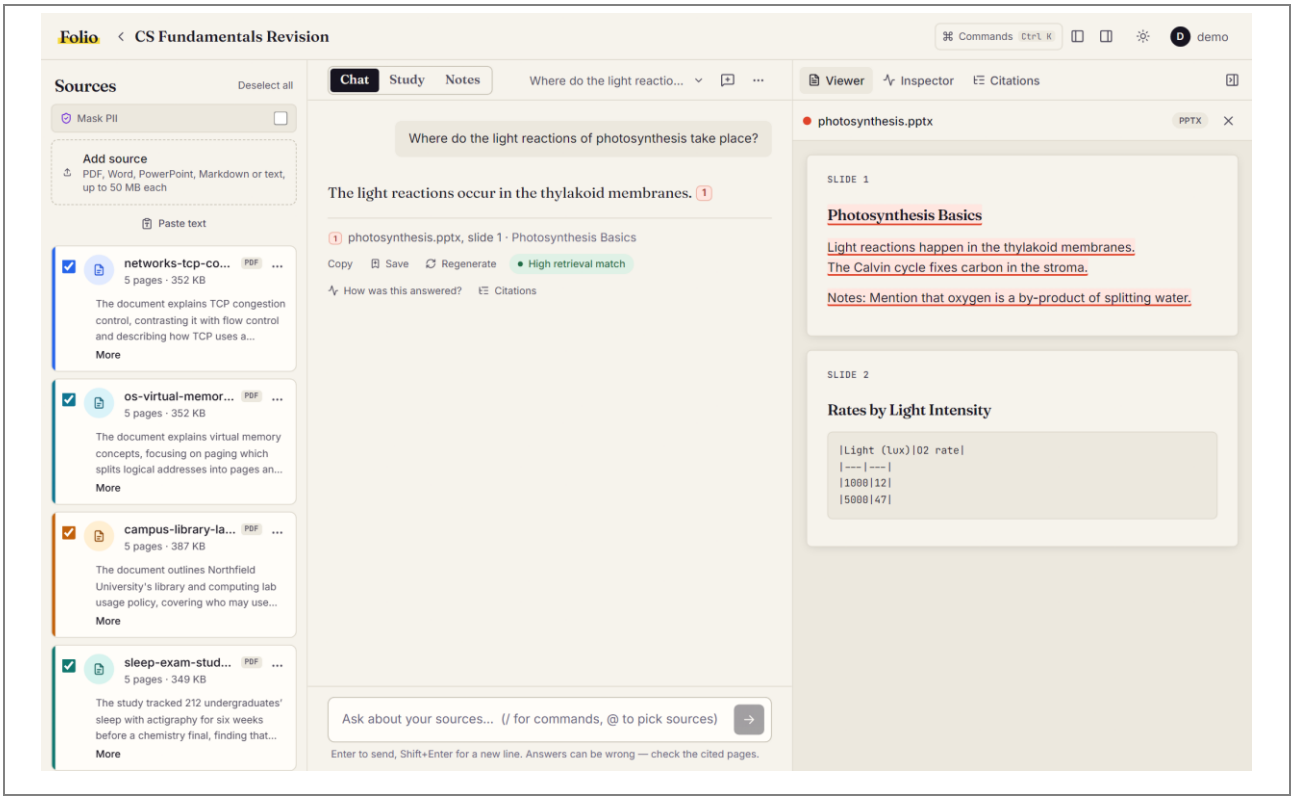


Fig. 10: PowerPoint citation opened in the text reader at the cited slide

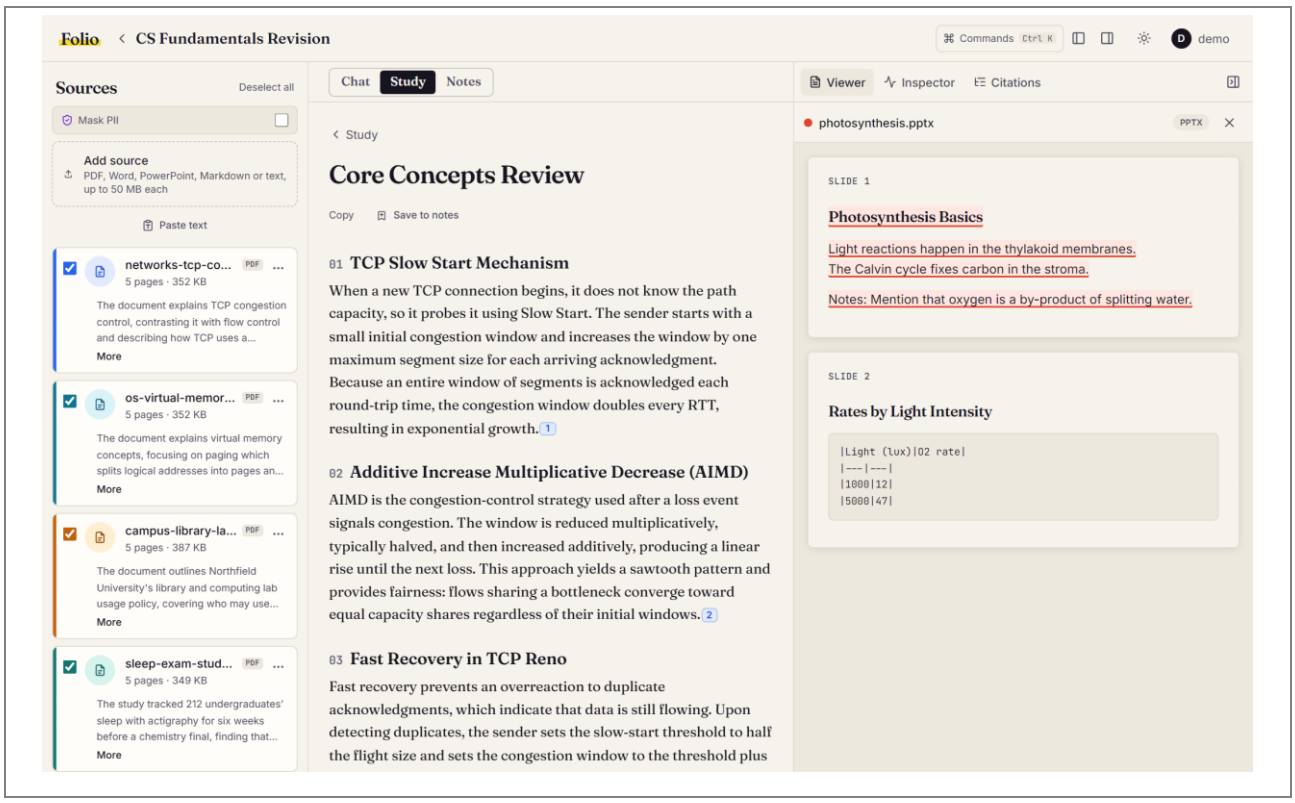


Fig. 11: Study guide generated from the sources, every section cited

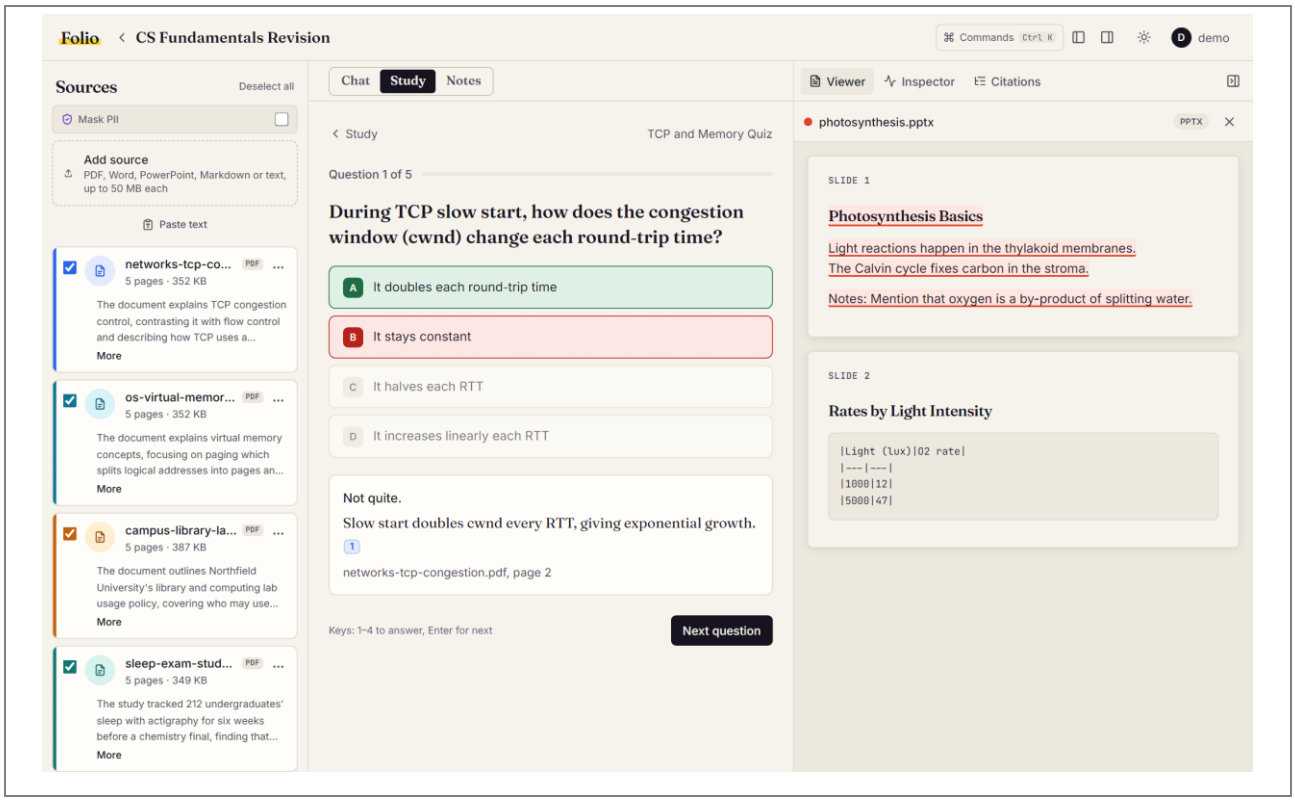


Fig. 12: Quiz with feedback, explanation and citation

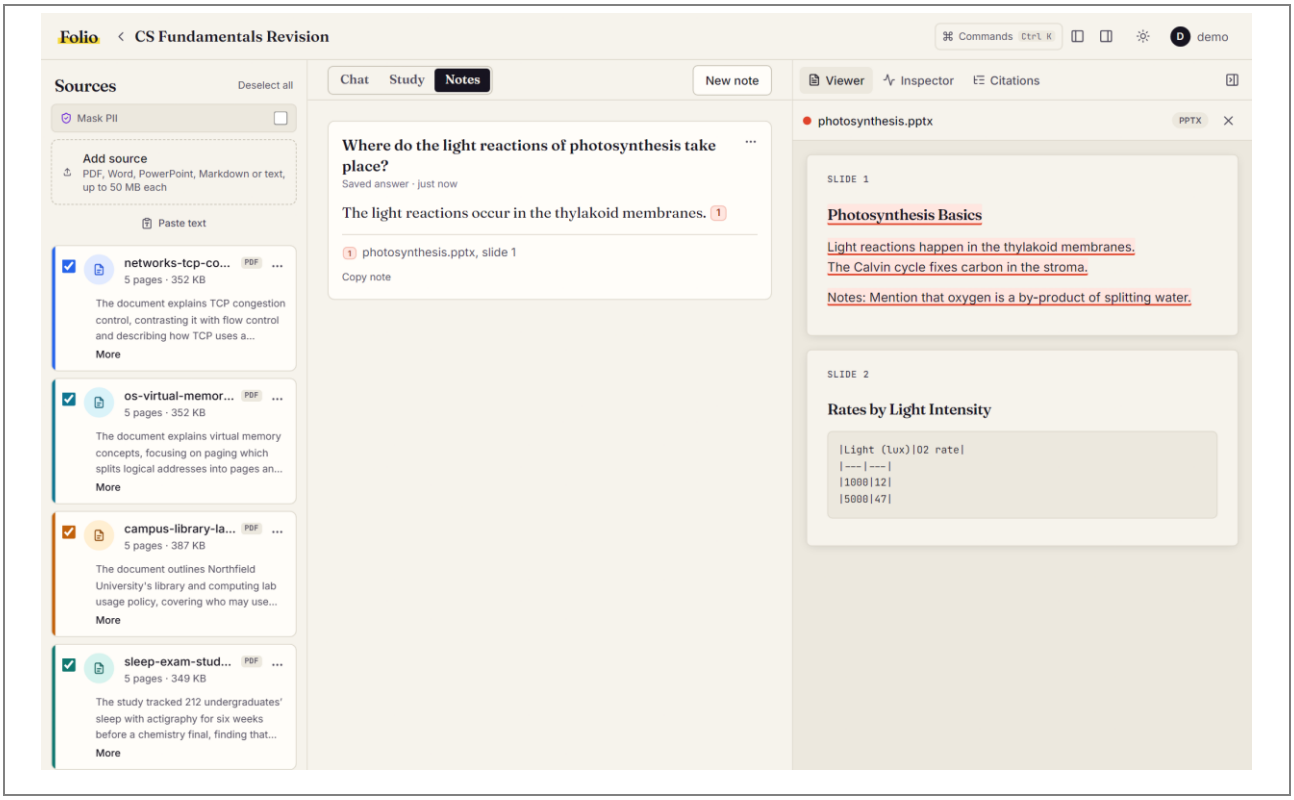


Fig. 13: Answer saved to Notes with a working citation

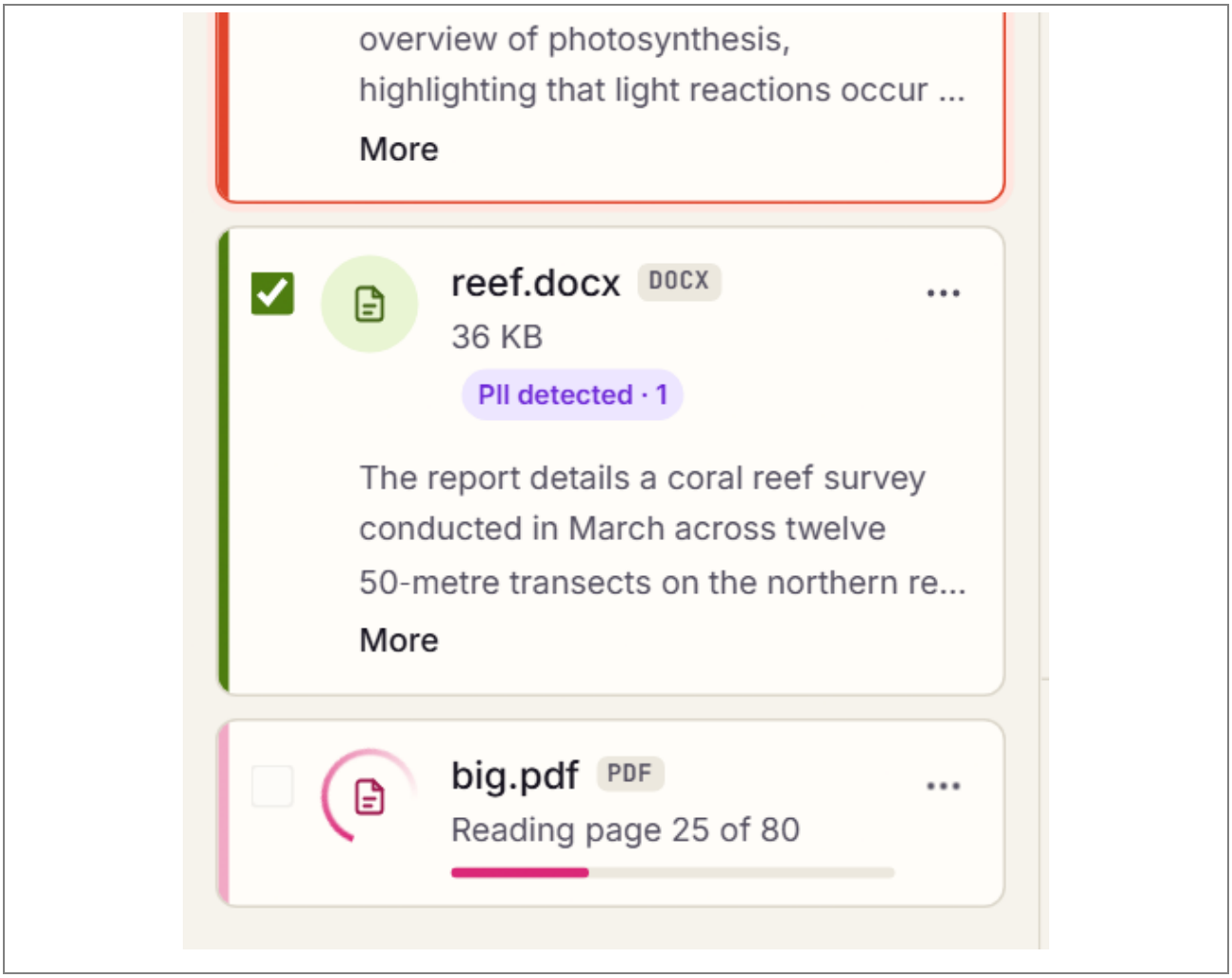


Fig. 14: Live indexing progress for an 80-page PDF

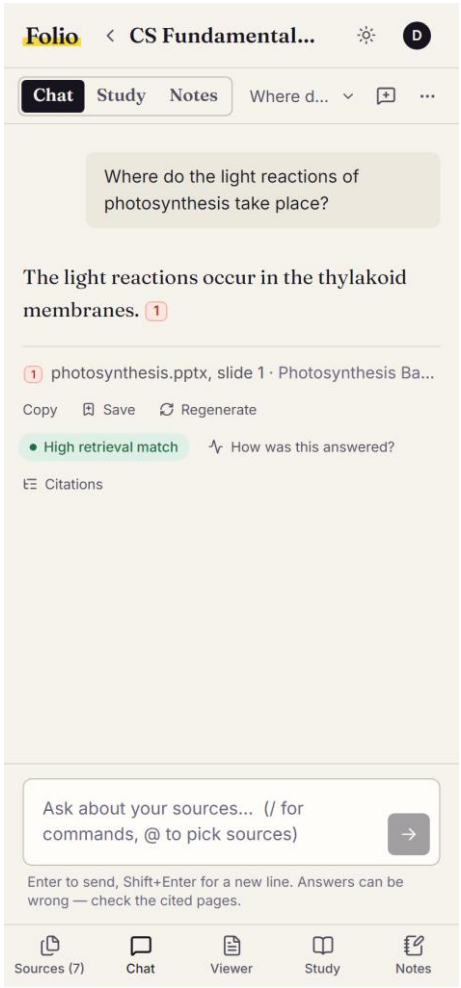


Fig. 15: Workspace on a phone-sized screen

10. TESTING AND RESULTS

10.1 Test Cases

Table 5: Test cases

Test ID	Description	Input	Expected Output	Actual Output	Status
TC-01	Upload each supported kind	DOCX, PPTX, MD, TXT files and pasted text	All reach ready; page text served	All 5 ready; chunk offsets matched page text exactly	Pass
TC-02	Reject a fake Word file	Random bytes named .docx	400 with clear message	400 "That file isn't a valid Word document."	Pass
TC-03	Reject a zip bomb	.docx declaring 210 MB uncompressed	400 before processing	400 "expands to too much data to process safely"	Pass
TC-04	Reject binary text and unsupported types	Binary data named .txt; a .png file	400 errors	400 "isn't plain text"; 400 listing supported types	Pass

Test ID	Description	Input	Expected Output	Actual Output	Status
TC-05	Citation opens the source	Click a citation in an answer	Viewer opens with cited text highlighted	Highlighted for PDF, PPTX, DOCX, MD, TXT and pasted text	Pass
TC-06	Decline unanswerable questions	"What is the capital city of Peru?"	Abstain, nothing cached	Abstained with nearest passages; not cached	Pass
TC-07	Answer cache	Same question twice, then after re-indexing	Second reply cached; miss after re-index	Cache hit in 156 ms; miss after re-index	Pass
TC-08	PII masking keeps highlights aligned	Masking on, re-index a DOCX with an email	Email masked, offsets valid	"[REDACTED EMAIL]" stored; 0 offset mismatches	Pass
TC-09	Summary failure does not break a source	Summary run with an invalid API key	Source stays ready	Source ready; summary status failed	Pass
TC-10	Data isolation	Request another user's thread id	404	404 Chat not found	Pass
TC-11	Regenerate and threads	Regenerate the latest answer; press New chat repeatedly	Fresh answer; one empty chat reused	New answer replaced old; one empty chat	Pass
TC-12	Responsive layout	Phone 390 px, tablet 768 px, 1024 x 768 and 1280 x 720	No horizontal overflow	No overflow on chat, viewer, study and notes	Pass

10.2 Performance Metrics

Table 6: Performance metrics of the final pipeline

Metric	Value	Method of Measurement
Hit@1 (right page ranked first)	93% (baseline 77%)	Evaluation harness, 43 answerable questions; gold pages
Hit@3 / Hit@6	95% / 100%	Evaluation harness, same questions
Mean reciprocal rank (MRR)	0.948 (baseline 0.852)	Evaluation harness
Unanswerable refused by the gate	57% (4 of 7), 0% of answerable refused	Confidence gate on 7 unanswerable questions
Citation precision	96%	Cited pages that are gold pages
Faithfulness	97%	Claims supported by the cited excerpts, judged by gpt-oss-20b
Key-fact recall	95%	Key facts stated in the answer, judged by gpt-oss-20b
Unanswerable handled	100%	Refused by the gate or declined by the model
Retrieval latency (median)	about 2.0 s on CPU	Embed + search + rerank, measured by the harness

Metric	Value	Method of Measurement

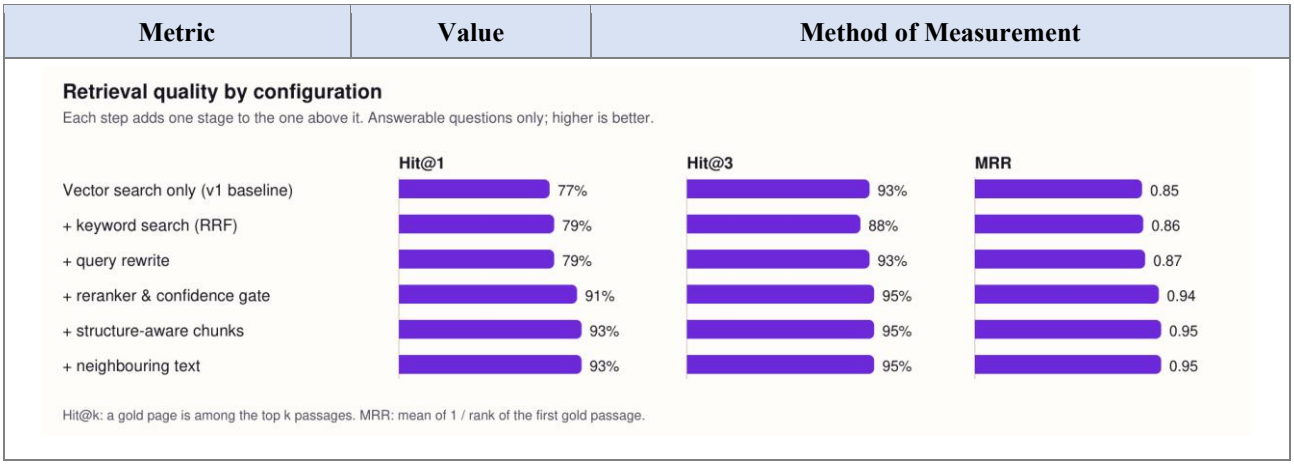


Fig. 16: Retrieval quality by configuration (evaluation harness)

10.3 Discussion of Results

Each retrieval stage was added one at a time. Keyword search with reciprocal rank fusion helped keyword questions (Hit@3 89% to 100%) but slightly hurt follow-ups; query rewriting restored follow-ups (40% to 80%). The cross-encoder reranker gave the largest single gain, raising Hit@1 from 79% to 91% and enabling the confidence gate, which refused 4 of 7 unanswerable questions; in the final configuration it refused no answerable question. Structure-aware chunks lifted Hit@1 to 93%.

Multi-query paraphrasing was measured and switched off by default: with the reranker in place it did not improve Hit@3 and lowered refusals of unanswerable questions (43% instead of 57%), while adding an LLM call. Neighbouring-chunk context does not change which page is retrieved; it gives the model fuller passages. With the full pipeline, answers reached 96% citation precision, 97% faithfulness and 95% key-fact recall.

Limitations: the corpus is small (one question moves a percentage by two to three points); the answer judge is itself a language model; reranking on the CPU is the slowest local step; and the three unanswerable questions that pass the gate have look-alike passages, so the model's own refusal is still needed.

11. CONCLUSION AND FUTURE WORK

11.1 Conclusion

Folio shows that a NotebookLM-style assistant with verifiable citations can be built from open components and free services and run on a single laptop. Every answer is grounded in the user's documents, every claim opens at its source location, and questions the sources cannot answer are declined. Measured stage by stage, hybrid search, query rewriting, cross-encoder reranking and structure-aware chunking raised Hit@1 from 77% to 93% and MRR from 0.852 to 0.948.

Beyond question answering, the system supports mixed source formats, multiple chat threads, notes and cited study material, with security controls for untrusted documents.

11.2 Future Work

The following extensions are planned:

- Add web pages, YouTube transcripts and audio as sources, and multilingual retrieval.
- Run embeddings and reranking on the GPU to cut retrieval latency, and move ingestion to a durable worker queue.
- Replace the relevance-based citation check with claim-level entailment and automatic answer repair.
- Larger and noisier evaluation sets with human judgements, and sharing of notebooks between users.

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9459–9474.

[2] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense passage retrieval for open-domain question answering," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 6769–6781.

[3] R. Nogueira and K. Cho, "Passage re-ranking with BERT," arXiv preprint arXiv:1901.04085, 2019.

[4] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, "C-Pack: Packed resources for general Chinese embeddings," in Proc. 47th Int. ACM SIGIR Conf. Research and Development in Information Retrieval, 2024, pp. 641–649.

[5] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling large language models to generate text with citations," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2023, pp. 6465–6488.

[6] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval, 2009, pp. 758–759.

[7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, 2019, pp. 3982–3992.

[8] Google, "NotebookLM." [Online]. Available: https://notebooklm.google.com

APPENDIX – SOURCE CODE

```
Full source code: https://github.com/swayam5342/Folio
Key excerpts (unchanged from the repository):

# ---- Reciprocal rank fusion and hybrid search (backend/app/services/retrieval.py)
def reciprocal_rank_fusion[K: Hashable](rankings: Sequence[Sequence[K]], k: int = RRF_K) -> list[tuple[K,
float]]:
    """Merge several best-first rankings into one: each item scores sum(1 / (k + rank)).

    Items found by more than one ranking rise to the top. Ties keep first-seen order,
    so the result is deterministic. Returns [(item, score)] best first."""
    scores: dict[K, float] = {}
    for ranked in rankings:
        for rank, item in enumerate(ranked, start=1):
            scores[item] = scores.get(item, 0.0) + 1.0 / (k + rank)
    return sorted(scores.items(), key=lambda pair: pair[1], reverse=True)

# ---- Confidence gate (backend/app/services/pipeline.py)
def assess_confidence(candidates: Sequence[Candidate], reranked: bool) -> tuple[Confidence | None, float
| None]:
    """(level, score behind it) from the best candidate; (None, None) when nothing was found.

    Cross-encoders are sharply polarised: a correct passage worded differently from the
    question can score near 0. So "low" needs the embedding similarity to be weak as well;
    a strong reranker score alone earns "high". The returned score is the reranker's when
    reranked, else the similarity."""
    if not candidates:
        return None, None
    similarity = max(c.vector_similarity or 0.0 for c in candidates)
    weak_similarity = similarity < settings.SIMILARITY_LOW
    if reranked:
        top = max(c.rerank_score or 0.0 for c in candidates)
        if top >= settings.CONFIDENCE_HIGH:
            level = "high"
        elif top < settings.CONFIDENCE_LOW and weak_similarity:
```

```
            level = "low"
        else:
            level = "medium"
    else:
        top = similarity
        level = "high" if similarity >= settings.SIMILARITY_HIGH else "low" if weak_similarity else
"medium"
    return level, round(top, 4)

def should_abstain(confidence: Confidence | None, broad: bool = False) -> bool:
    """Refuse to answer on low confidence, unless the question is about the sources as a whole."""
    return settings.ENABLE_ABSTENTION and confidence == "low" and not broad

# ---- Neighbouring-chunk expansion (backend/app/services/pipeline.py)
async def expand_neighbors(db: AsyncSession, selected: list[dict]) -> None:
    """Attach the chunks just before and after each selected passage as item["before"] /
    item["after"], so the model reads whole thoughts rather than cut-off excerpts.

    A neighbour must be on the same page and in the same section (so the citation's page
    stays exact), not quarantined, not selected itself, not used for another passage and
    not overlapping the passage's text. Best-ranked passages are expanded first, while
    the context stays within CONTEXT_MAX_CHARS."""
    wanted = {
        (item["chunk"].document_id, item["chunk"].chunk_index + delta)
        for item in selected
        for delta in (-1, 1)
    }
    rows = (await db.scalars(
        select(Chunk).where(
            tuple_(Chunk.document_id, Chunk.chunk_index).in_(list(wanted)),
            Chunk.quarantined.is_(False),
        )
    )).all()
    by_position = {(c.document_id, c.chunk_index): c for c in rows}
    used = {item["chunk"].id for item in selected}
    total = sum(len(item["chunk"].content) for item in selected)
    for item in selected:  # already best first
        chunk = item["chunk"]
        for key, delta in (("before", -1), ("after", 1)):
            neighbour = by_position.get((chunk.document_id, chunk.chunk_index + delta))
            if (
                neighbour is None
                or neighbour.id in used
                or neighbour.page_number != chunk.page_number
                or neighbour.section_title != chunk.section_title
                or _overlaps(neighbour, chunk)
                or total + len(neighbour.content) > settings.CONTEXT_MAX_CHARS
            ):
                continue
            item[key] = neighbour.content
            used.add(neighbour.id)
            total += len(neighbour.content)

# ---- Prompt construction with labelled excerpts (backend/app/services/llm.py)
def build_context(retrieved: list[dict]) -> tuple[str, dict]:
    context_blocks = []
    citation_map = {}

    for i, item in enumerate(retrieved, start=1):
        chunk = item["chunk"]
        document = item["document"]
        kind = getattr(document, "kind", None) or "pdf"
        candidate = item.get("candidate")  # set by the retrieval pipeline; absent for study sets
        relevance = None
        if candidate is not None:
            relevance = candidate.rerank_score if candidate.rerank_score is not None else
candidate.vector_similarity
        marker = f"C{i}"
        citation_map[marker] = {
            "marker": marker,
            "chunk_id": str(chunk.id),
            "document_id": str(document.id),
            "filename": document.filename,
            "document_kind": kind,
```

```
            "page_number": chunk.page_number,
            "section_title": getattr(chunk, "section_title", None),
            "char_start": getattr(chunk, "char_start", None),
            "char_end": getattr(chunk, "char_end", None),
            "snippet": chunk.content[:240],
            "score": None if relevance is None else float(f"{relevance:.4g}"),
            # Embedding similarity of question and passage: still meaningful for broad questions,
            # where no single passage "answers" the question and the cross-encoder scores ~0.
            "similarity": None if candidate is None or candidate.vector_similarity is None else
round(candidate.vector_similarity, 4),
        }
        where = ", ".join(
            filter(None, [f'from "{document.filename}"', location(kind, chunk.page_number,
citation_map[marker]["section_title"])])
        )
        context_blocks.append(f"<source id=\"{marker}\"
location={where!r}>\n{passage_text(item)}\n</source>")

    return "\n\n---\n\n".join(context_blocks), citation_map


def parse_citations(answer_text: str, citation_map: dict) -> list[dict]:
    """answer_text must already be normalized with normalize_citations."""
    used_markers = sorted(set(CITATION_PATTERN.findall(answer_text)), key=int)
    return [citation_map[f"C{n}"] for n in used_markers if f"C{n}" in citation_map]

# ---- Citation coverage (backend/app/services/citation_check.py)
def citation_coverage(answer: str, valid_markers: set[str]) -> tuple[float, list[str]]:
    """Fraction of substantive answer claims carrying at least one valid citation.

    As in claims_by_marker, markers placed after the full stop ("... bleached. [[C1]]")
    split off as a sentence of their own and count for the claim before them."""
    total = 0
    cited = 0
    uncited: list[str] = []
    last_uncited = False  # whether the claim just before this sentence lacked a citation
    for sentence in _SPLIT.split(answer):
        text = _DECORATION.sub("", CITATION_PATTERN.sub("", sentence)).strip()
        markers = {f"C{n}" for n in CITATION_PATTERN.findall(sentence)}
        if not text and last_uncited and markers & valid_markers:
            uncited.pop()  # it was uncited only because its markers came after the full stop
            cited += 1
            last_uncited = False
            continue
        if len(text) < MIN_CLAIM_CHARS:
            continue
        # Headings and explicit uncertainty do not assert a source fact.
        if sentence.lstrip().startswith("#") or text.casefold().startswith(("i couldn't", "the sources
don't")):
            continue
        total += 1
        last_uncited = not markers & valid_markers
        if last_uncited:
            uncited.append(text)
        else:
            cited += 1
    return (1.0 if total == 0 else cited / total), uncited


# ---- Answer-cache key (backend/app/services/answer_cache.py)
def cache_key(standalone: str, document_ids: Sequence[uuid.UUID] | None, source_fingerprint: str) -> str:
    scope = "all" if document_ids is None else sorted(str(d) for d in document_ids)
    material = {
        "q": normalize_question(standalone),
        "scope": scope,
        "sources": source_fingerprint,
        "model": settings.GROQ_MODEL,
        "flags": settings.feature_flags(),
    }
    return hashlib.sha256(json.dumps(material, sort_keys=True).encode()).hexdigest()
```